

Ray: Your Gateway to Scalable AI and Machine Learning Applications



Krishnaveni Ponna

Last Updated : 25 Mar, 2025



Ray has emerged as a powerful framework for distributed computing in AI and ML workloads, enabling researchers and practitioners to scale their applications from laptops to clusters with minimal code changes. This guide provides an in-depth exploration of Ray's architecture, capabilities, and applications in modern [machine learning](#) workflows, complete with a practical project implementation.

Learning Objectives

- Understand Ray's architecture and its role in distributed computing for AI/ML.
- Leverage Ray's ecosystem (Train, Tune, Serve, Data) for end-to-end ML workflows.
- Compare Ray with alternative distributed computing frameworks.
- Design distributed training pipelines for [large language models](#).
- Optimize resource allocation and debug distributed applications.

This article was published as a part of the [Data Science Blogathon](#).

Table of contents

1. Introduction to Ray and Distributed Computing

2. Challenge of Scaling Python Applications
3. Ray Framework
4. Getting Started with Ray
5. Ray's Programming Model: Tasks and Actors
6. Ray Cluster Architecture
7. Ray Object Store and Memory Management

Introduction to Ray and Distributed Computing

Ray is an open-source unified framework for scaling AI and [Python applications](#), providing a simple, universal API for building distributed applications that can scale from a laptop to a cluster. Developed originally at UC Berkeley's RISELab and now maintained by [Anyscale](#), Ray has gained significant traction in the AI community, becoming the backbone for training and deploying some of the most advanced AI models today.

New Feature



Get Personalized Learning Path!

Set your goal and timeline. Get a path—under 2 mins.

Create My Path

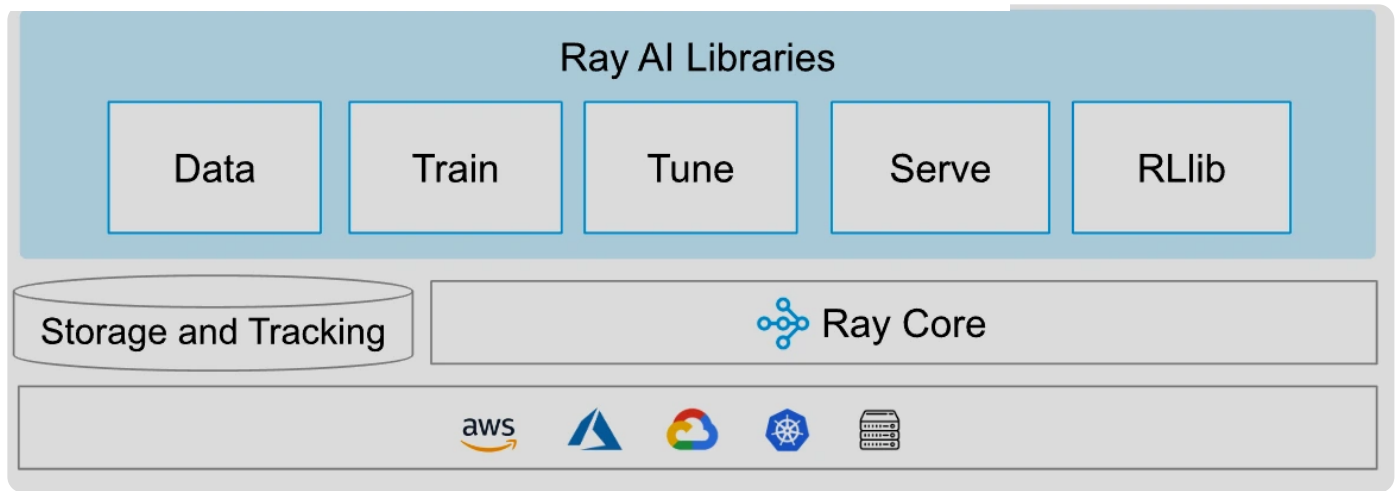


Image Source: [Click Here](#)

The growing importance of distributed computing in AI stems from several factors:

- **Increasing model sizes:** Modern AI models, especially large language models (LLMs), have grown exponentially in size, with billions or even trillions of parameters.
- **Expanding datasets:** Training data continues to grow in volume, often exceeding what can be processed on a single machine.
- **Computational demands:** Complex algorithms and training procedures require more computational resources than individual machines can provide.
- **Deployment challenges:** Serving models at scale requires distributed infrastructure to handle varying workloads efficiently.

Traditional distributed computing frameworks often require significant rewrites of existing code, presenting a steep learning curve. Ray differentiates itself by offering a simple, intuitive API that makes transitioning from single-machine to multi-machine computation straightforward, often requiring only a few decorator changes to existing Python code.

Challenge of Scaling Python Applications

Python has become the lingua franca of data science and machine learning, but it wasn't designed with distributed computing in mind. When practitioners need to scale their Python applications, they traditionally face several challenges:

- **Low-level distribution concerns:** Managing worker processes, load balancing, and fault tolerance.
- **Data movement:** Efficiently transferring data between machines.
- **Resource management:** Allocating and tracking CPU, GPU, and memory resources across a cluster.
- **Code complexity:** Rewriting algorithms to work in a distributed fashion.

It addresses these challenges by providing a unified framework that abstracts away much of the complexity while still allowing fine-grained control when needed.

Ray Framework

Ray Framework architecture is structured into three primary components:

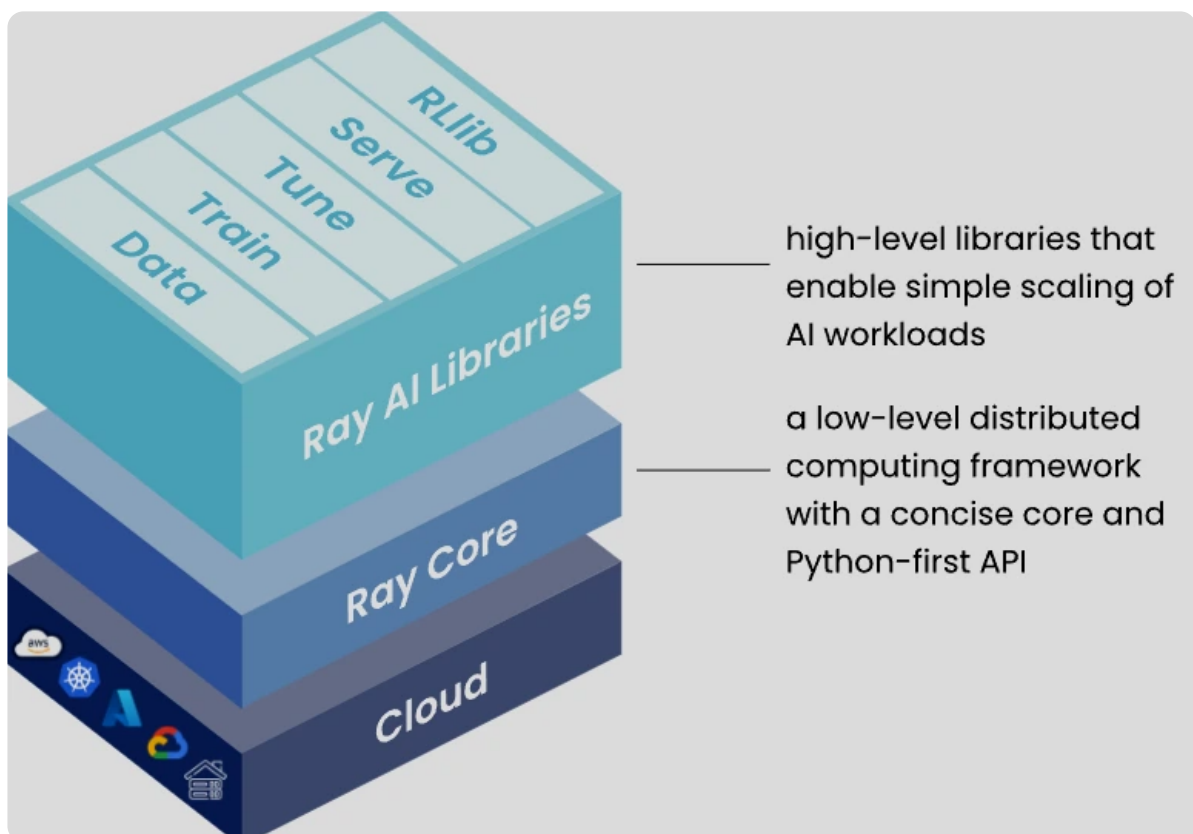


Image Source: [Click Here](#)

- **Ray AI Libraries:** This collection of Python-based, domain-specific libraries provides machine learning engineers, data scientists, and researchers with a scalable toolkit tailored for various ML applications.
- **Ray Core:** Serving as the foundation, Ray Core is a general-purpose distributed computing library that empowers Python developers to parallelize and scale applications, thereby enhancing machine learning workloads.
- **Ray Clusters:** Comprising multiple worker nodes linked to a central head node, Ray Clusters can be configured with a fixed size or set to dynamically adjust resources based on the demands of the running applications.

This modular design enables users to efficiently build and manage distributed applications without requiring in-depth expertise in distributed systems.

Getting Started with Ray

Before diving into the advanced applications, it's essential to set up your [Ray environment](#) and understand the basics of getting started.

Ray can be installed using pip. To install the latest stable version, run:

```
# For machine learning applications
```

Copy Code

```
pip install -U "ray[data,train,tune,serve]"
```

```
## For reinforcement learning support, install RLlib instead.
```

```
## pip install -U "ray[rllib]"
```

```
# For general Python applications
```

```
pip install -U "ray[default]"
```

```
## If you don't want Ray Dashboard or Cluster Launcher, install Ray with minim
```

```
## pip install -U "ray"
```

```
2025-03-25 07:55:56,044 INFO worker.py:1852 -- Started a local Ray instance.
```



```
Python version: 3.11.11
```

```
Ray version: 2.44.0
```

Ray's Programming Model: Tasks and Actors

Ray's programming model revolves around two primary abstractions:

- **Tasks:** Functions that execute remotely and asynchronously. [Tasks](#) are stateless computations that can be scheduled on any worker in the cluster.
- **Actors:** Classes that maintain state and execute methods remotely. [Actors](#) encapsulate state and provide an object-oriented approach to distributed computing.

These abstractions allow developers to express different types of parallelism naturally:

```
import ray
# Initialize Ray
ray.init()

# Define a remote task
@ray.remote
def process_data(data_chunk):
    # Process data and return results
    return processed_result

# Define an actor class
@ray.remote
class Counter:
    def __init__(self):
        self.count = 0

    def increment(self):
        self.count += 1
        return self.count

    def get_count(self):
        return self.count
```

[Copy Code](#)

```
# Execute tasks in parallel
data_chunks = [data_1, data_2, data_3, data_4]
result_refs = [process_data.remote(chunk) for chunk in data_chunks]
results = ray.get(result_refs) # Wait for all tasks to complete

# Create an actor instance
counter = Counter.remote()
counter.increment.remote() # Execute method on the actor
count = ray.get(counter.get_count.remote()) # Get the actor's state
```

Ray's programming model makes it easy to transform sequential Python code into distributed applications with minimal changes. Tasks are ideal for stateless, embarrassingly parallel workloads, while actors are perfect for maintaining state or implementing services.

Ray Cluster Architecture

A Ray cluster consists of several key components:

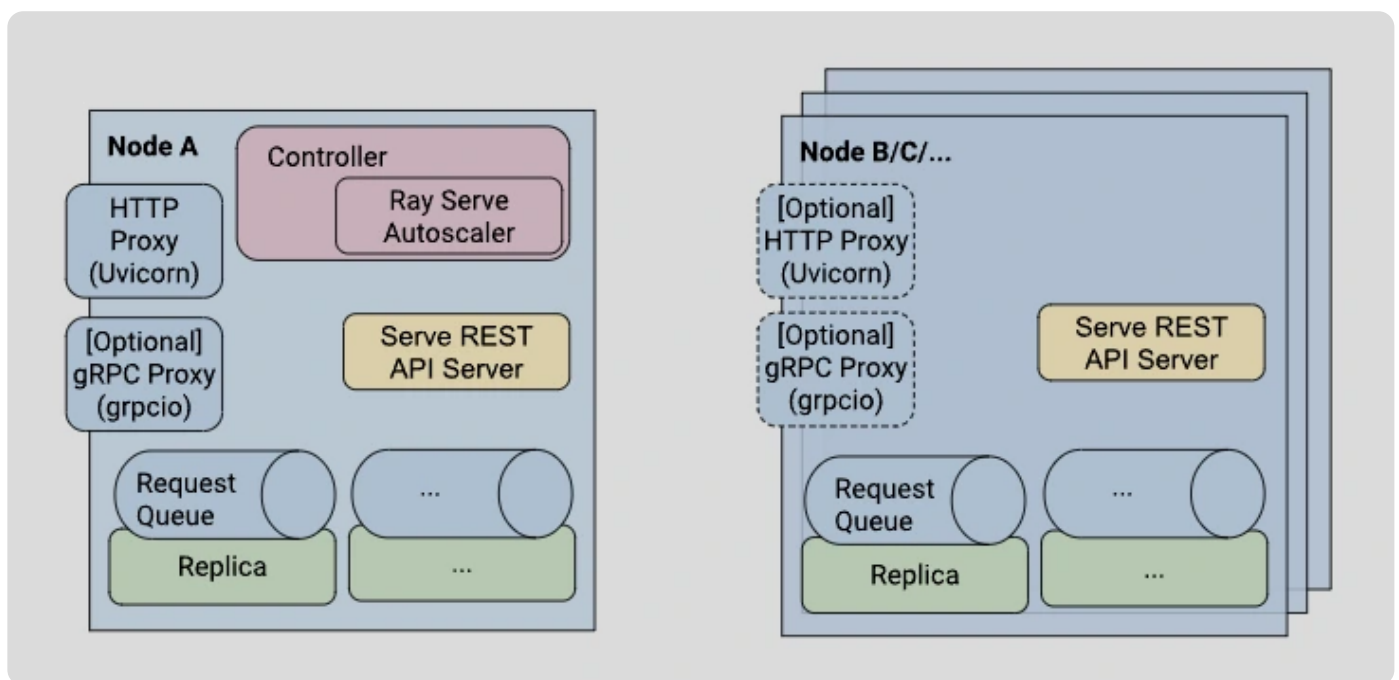


Image Source: [Click Here](#)

- **Head Node:** The central coordination point for the cluster, hosting the Global Control Store (GCS) which maintains cluster metadata.

- **Worker Nodes:** Processes that execute tasks and host actors. Each worker runs on a separate CPU or GPU core.
- **Driver Process:** The process running the user's program, responsible for submitting tasks to the cluster.
- **Object Store:** A distributed, shared-memory object store for efficient data sharing between tasks and actors.
- **Scheduler:** Responsible for assigning tasks to workers based on resource availability and constraints.
- **Resource Management:** Ray's system for allocating and tracking CPU, GPU, and custom resources across the cluster.

Setting up a Ray cluster can be done in multiple ways:

- Locally on a single machine
- On a private cluster using Ray's cluster launcher
- On cloud providers like AWS, GCP, or Azure
- Using managed services like Anyscale

```
# Starting Ray on a single machine (head node)
ray start --head --port=6379
```

[Copy Code](#)

```
# Joining a worker node to the cluster
ray start --address=<head_node_address>:6379
```

Ray Object Store and Memory Management

Ray includes a distributed object store that enables efficient sharing of objects between tasks and actors. Objects in the store are immutable and can be accessed by any worker in the cluster.


```
import ray
import numpy as np

ray.init()

# Store an object in the object store
data = np.random.rand(1000, 1000)
data_ref = ray.put(data) # Returns a reference to the object

# Pass the reference to a remote task
@ray.remote
def process_matrix(matrix_ref):
    # The matrix is retrieved from the object store
    matrix = ray.get(matrix_ref)
    return np.sum(matrix)

result_ref = process_matrix.remote(data_ref)
result = ray.get(result_ref)
```

The object store optimizes data transfer by:

- **Avoiding unnecessary data copying:** Objects are shared by reference when possible.
- **Spilling to disk:** Automatically moving objects to disk when memory is limited.
- **Distributed references:** Tracking object references across the cluster.

Ray for AI and ML Workloads

The Ray provides a comprehensive ecosystem of libraries specifically designed for different aspects of AI and ML workflows:

Ray Train for Distributed Model Training using PyTorch

Ray Train simplifies distributed deep learning with a unified API across different frameworks

For reference, the final code will look something like the following:

```
import os
import tempfile

import torch
from torch.nn import CrossEntropyLoss
from torch.optim import Adam
from torch.utils.data import DataLoader
from torchvision.models import resnet18
from torchvision.datasets import FashionMNIST
from torchvision.transforms import ToTensor, Normalize, Compose

import ray.train.torch

def train_func():
    # Model, Loss, Optimizer
    model = resnet18(num_classes=10)
    model.conv1 = torch.nn.Conv2d(
        1, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False
    )
    # [1] Prepare model.
    model = ray.train.torch.prepare_model(model)
    # model.to("cuda") # This is done by `prepare_model`
    criterion = CrossEntropyLoss()
    optimizer = Adam(model.parameters(), lr=0.001)

    # Data
    transform = Compose([ToTensor(), Normalize((0.28604, ), (0.32025, ))])
    data_dir = os.path.join(tempfile.gettempdir(), "data")
    train_data = FashionMNIST(root=data_dir, train=True, download=True, transf
    train_loader = DataLoader(train_data, batch_size=128, shuffle=True)
    # [2] Prepare dataloader.
    train_loader = ray.train.torch.prepare_data_loader(train_loader)

    # Training
    for epoch in range(10):
        if ray.train.get_context().get_world_size() > 1:
            train_loader.sampler.set_epoch(epoch)

        for images, labels in train_loader:
            # This is done by `prepare_data_loader`!
            # images, labels = images.to("cuda"), labels.to("cuda")
            outputs = model(images)
            loss = criterion(outputs, labels)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # [3] Report metrics and checkpoint.
        metrics = {"loss": loss.item(), "epoch": epoch}
```

```

    with tempfile.TemporaryDirectory() as temp_checkpoint_dir:
        torch.save(
            model.module.state_dict(),
            os.path.join(temp_checkpoint_dir, "model.pt")
        )
        ray.train.report(
            metrics,
            checkpoint=ray.train.Checkpoint.from_directory(temp_checkpoint_dir)
        )
    if ray.train.get_context().get_world_rank() == 0:
        print(metrics)

# [4] Configure scaling and resource requirements.
scaling_config = ray.train.ScalingConfig(num_workers=2, use_gpu=True)

# [5] Launch distributed training job.
trainer = ray.train.torch.TorchTrainer(
    train_func,
    scaling_config=scaling_config,
    # [5a] If running in a multi-node cluster, this is where you
    # should configure the run's persistent storage that is accessible
    # across all worker nodes.
    # run_config=ray.train.RunConfig(storage_path="s3://..."),
)
result = trainer.fit()

# [6] Load the trained model.
with result.checkpoint.as_directory() as checkpoint_dir:
    model_state_dict = torch.load(os.path.join(checkpoint_dir, "model.pt"))
    model = resnet18(num_classes=10)
    model.conv1 = torch.nn.Conv2d(
        1, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False
    )
    model.load_state_dict(model_state_dict)

```

Ray Train provides:

- Multi-node and multi-GPU training capabilities
- Support for popular frameworks (PyTorch, TensorFlow, Horovod)
- Checkpointing and fault tolerance
- Integration with hyperparameter tuning

Ray Tune for Hyperparameter Optimization

Hyperparameter tuning is crucial for AI and ML model performance. [Ray Tune](#) provides scalable hyperparameter optimization.

To run, install the following:

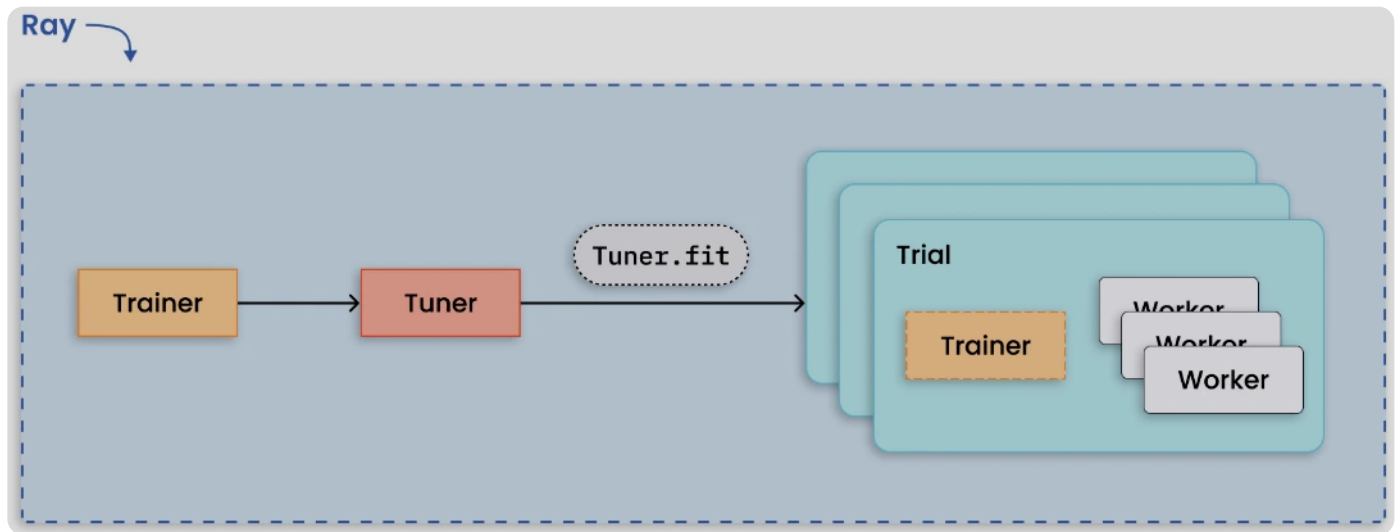


Image Source: [Click Here](#)

```
pip install "ray[tune]"
```

Copy Code

```
from ray import tune
from ray.tune.schedulers import ASHAScheduler

# Define the objective function to optimize
def objective(config):
    model = build_model(config)
    for epoch in range(100):
        # Train the model
        loss = train_epoch(model)
        tune.report(loss=loss) # Report metrics to Tune

# Configure the search space
search_space = {
    "learning_rate": tune.loguniform(1e-4, 1e-1),
    "batch_size": tune.choice([16, 32, 64, 128]),
    "hidden_layers": tune.randint(1, 5)
}

# Run hyperparameter optimization
analysis = tune.run(
    objective,
    config=search_space,
    scheduler=ASHAScheduler(metric="loss", mode="min"),
    num_samples=100
```

Copy Code

```
)
```

```
# Get the best configuration
best_config = analysis.get_best_config(metric="loss", mode="min")
```

Ray Tune offers:

- Various search algorithms (grid search, random search, Bayesian optimization)
- Adaptive resource allocation
- Early stopping for inefficient trials
- Integration with ML frameworks

Ray Serve for Model Deployment

It is designed for deploying ML models at scale:

Install Ray Serve and its dependencies:

```
#import csv
```

[Copy Code](#)

```
import ray
from ray import serve
from starlette.requests import Request
import torch
import json

# Start Ray Serve
serve.start()

# Define a deployment for our model
@serve.deployment(route_prefix="/predict", num_replicas=2)
class ModelDeployment:
    def __init__(self, model_path):
        self.model = torch.load(model_path)
        self.model.eval()

    async def __call__(self, request: Request):
        data = await request.json()
        input_tensor = torch.tensor(data["input"])
```

[Copy Code](#)

```
with torch.no_grad():
    prediction = self.model(input_tensor).tolist()

    return {"prediction": prediction}

# Deploy the model
model_deployment = ModelDeployment.deploy("./trained_model.pt")
```

The Ray Serve enables:

- Model composition and microservices
- Horizontal scaling
- Traffic splitting and A/B testing
- Batching for performance optimization

Ray Data for ML-Optimized Data Processing

Ray Data provides distributed data processing capabilities optimized for ML workloads:

```
import ray

# Initialize Ray
ray.init()

# Create a dataset from a file or data source
ds = ray.data.read_csv("s3://bucket/path/to/data.csv")

# Apply transformations in parallel
def preprocess_batch(batch):
    # Apply preprocessing to the batch
    return processed_batch

transformed_ds = ds.map_batches(preprocess_batch)

# Split for training and validation
train_ds, val_ds = transformed_ds.train_test_split(test_size=0.2)

# Create a loader for ML framework (e.g., PyTorch)
train_loader = train_ds.to_torch(batch_size=32, shuffle=True)
```

[Copy Code](#)

Data offers:

- Parallel data loading and transformation
- Integration with ML training
- Support for various data formats and sources
- Optimized for ML workflows

Distributed Fine-tuning of a Large Language Model with Ray

Let's implement a complete project that demonstrates how to use Ray for fine-tuning a [large language model](#) (LLM) using distributed computing resources. We'll use GPT-J-6B as our base model and Ray Train with DeepSpeed for efficient distributed training.

In this project, we will:

- Set up a Ray cluster for distributed training
- Prepare a dataset for fine-tuning the LLM
- Configure DeepSpeed for memory-efficient training
- Implement distributed training using Ray Train
- Evaluate the model and deploy it with Ray Serve

Environment Setup

First, let's set up our environment with the necessary dependencies:

```
# Install required packages
!pip install "ray[train]" transformers datasets accelerate deepspeed torch eva
```

[Copy Code](#)

Ray Cluster Configuration

For this project, we'll configure a Ray cluster with multiple GPUs:

[Copy Code](#)

```
import ray
import os

# Configuration
model_name = "EleutherAI/gpt-j-6B" # We'll use GPT-J-6B as our base model
use_gpu = True
num_workers = 16 # Number of training workers (adjust based on available GPUs)
cpus_per_worker = 8 # CPUs per worker

# Initialize Ray
ray.init(
    runtime_env={
        "pip": [
            "transformers==4.26.0",
            "accelerate==0.18.0",
            "datasets",
            "evaluate",
            "deepspeed==0.12.3",
            "torch>=1.12.0"
        ]
    }
)
```

This initialization creates a local Ray cluster. In a production environment, you might connect to an existing Ray cluster instead.

Data Preparation

For fine-tuning our language model, we'll prepare a text dataset:

[Copy Code](#)

```
from datasets import load_dataset
from transformers import AutoTokenizer

# Load tokenizer for our model
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token # GPT models don't have a pad token

# Load a text dataset (example using a subset of wikitext)
dataset = load_dataset("wikitext", "wikitext-2-raw-v1")

# Define preprocessing function for tokenization
```



```
def preprocess_function(examples):
    return tokenizer(
        examples["text"],
        truncation=True,
        max_length=512,
        padding="max_length",
        return_tensors="pt"
    )

# Tokenize the dataset in parallel using Ray Data
import ray.data
ray_dataset = ray.data.from_huggingface(dataset)
tokenized_dataset = ray_dataset.map_batches(
    preprocess_function,
    batch_format="pandas",
    batch_size=100
)

# Convert back to Hugging Face dataset format
train_dataset = tokenized_dataset.train.to_huggingface()
eval_dataset = tokenized_dataset.validation.to_huggingface()
```



DeepSpeed Configuration for Memory-Efficient Training

Training large models like GPT-J-6B requires memory optimization techniques.

[DeepSpeed](#) is a deep learning optimization library that enables efficient training.

Let's configure it for our distributed training:

```
# DeepSpeed configuration
deepspeed_config = {
    "fp16": {
        "enabled": True
    },
    "zero_optimization": {
        "stage": 2,
        "offload_optimizer": {
            "device": "cpu"
        },
        "allgather_bucket_size": 5e8,
        "reduce_bucket_size": 5e8
    },
    "train_batch_size": "auto",
    "train_micro_batch_size_per_gpu": 4,
```

[Copy Code](#)

```
"gradient_accumulation_steps": "auto",
"optimizer": {
    "type": "AdamW",
    "params": {
        "lr": 5e-5,
        "weight_decay": 0.01
    }
}

# Save the config to a file
import json
with open("deepspeed_config.json", "w") as f:
    json.dump(deepspeed_config, f)
```

This configuration uses several optimization techniques:

- FP16 precision to reduce memory usage
- ZeRO stage 2 optimizer to partition optimizer states
- CPU offloading to move some data from GPU to CPU memory
- Automatic batch size and gradient accumulation configuration

Implementing Distributed Training

Define the training function and use Ray Train to distribute it across the cluster:

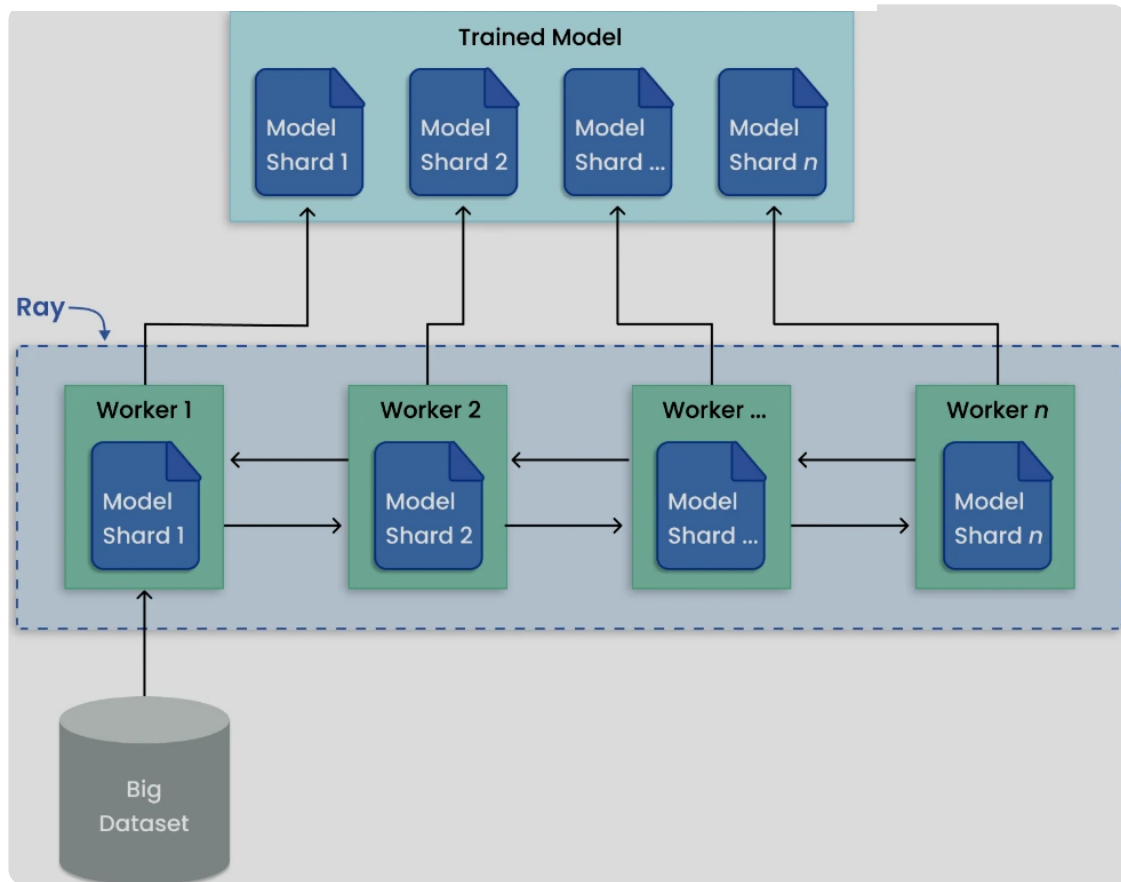


Image Source: [Click Here](#)

```
from transformers import AutoModelForCausalLM, Trainer, TrainingArguments
import torch
import torch.distributed as dist
from ray.train.huggingface import HuggingFaceTrainer
from ray.train import ScalingConfig

# Define the training function to be executed on each worker
def train_func(config):
    # Initialize process group for distributed training
    dist.init_process_group(backend="nccl")

    # Load pre-trained model
    model = AutoModelForCausalLM.from_pretrained(
        config["model_name"],
        revision="float16",
        torch_dtype=torch.float16,
        low_cpu_mem_usage=True
    )

    # Set up training arguments
    training_args = TrainingArguments(
        output_dir="./output",
        per_device_train_batch_size=config["batch_size"],
        per_device_eval_batch_size=config["batch_size"],
        evaluation_strategy="epoch",
```

[Copy Code](#)

```
        num_train_epochs=config["epochs"],
        fp16=True,
        report_to="none",
        deepspeed="deepspeed_config.json",
        save_strategy="epoch",
        load_best_model_at_end=True,
        logging_steps=10
    )

    # Initialize Trainer
    trainer = Trainer(
        model=model,
        args=training_args,
        train_dataset=config["train_dataset"],
        eval_dataset=config["eval_dataset"],
    )

    # Train the model
    trainer.train()

    # Save the final model
    trainer.save_model("./final_model")

    return {"loss": trainer.state.best_metric}

# Configure the distributed training
scaling_config = ScalingConfig(
    num_workers=num_workers,
    use_gpu=use_gpu,
    resources_per_worker={"CPU": cpus_per_worker, "GPU": 1}
)

# Create the Ray Train Trainer
trainer = HuggingFaceTrainer(
    train_func,
    scaling_config=scaling_config,
    train_loop_config={
        "model_name": model_name,
        "train_dataset": train_dataset,
        "eval_dataset": eval_dataset,
        "batch_size": 4,
        "epochs": 3
    }
)

# Start the distributed training
result = trainer.fit()
```

This code sets up distributed training across multiple GPUs using Ray Train.

The `train_func` is executed on each worker, with Ray handling the distribution of the workload.

Model Evaluation

After training, we'll evaluate the model's performance:

```
from transformers import pipeline

# Load the fine-tuned model
model_path = "./final_model"
tokenizer = AutoTokenizer.from_pretrained(model_path)
model = AutoModelForCausalLM.from_pretrained(model_path)

# Create a text generation pipeline
text_generator = pipeline("text-generation", model=model, tokenizer=tokenizer,

# Example prompts for evaluation
prompts = [
    "Artificial intelligence is",
    "The future of distributed computing",
    "Machine learning models can"
]

# Generate text for each prompt
for prompt in prompts:
    generated_text = text_generator(prompt, max_length=100, num_return_sequences=1)
    print(f"Prompt: {prompt}")
    print(f"Generated: {generated_text}")
    print("---")
```

[Copy Code](#)

Deploying the Model with Ray Serve

Finally, we'll deploy the fine-tuned model for inference using Ray Serve:

```
import ray
from ray import serve
from starlette.requests import Request
import json
```

[Copy Code](#)

```

# Start Ray Serve
serve.start()

# Define a deployment for our model
@serve.deployment(route_prefix="/generate", num_replicas=2, ray_actor_options=
class TextGenerationModel:
    def __init__(self, model_path):
        self.tokenizer = AutoTokenizer.from_pretrained(model_path)
        self.model = AutoModelForCausalLM.from_pretrained(
            model_path,
            torch_dtype=torch.float16,
            device_map="auto"
        )
        self.pipeline = pipeline(
            "text-generation",
            model=self.model,
            tokenizer=self.tokenizer
        )

    async def __call__(self, request: Request) -> dict:
        data = await request.json()
        prompt = data.get("prompt", "")
        max_length = data.get("max_length", 100)

        generated_text = self.pipeline(
            prompt,
            max_length=max_length,
            num_return_sequences=1
        )[0]["generated_text"]

        return {"generated_text": generated_text}

# Deploy the model
model_deployment = TextGenerationModel.deploy("./final_model")

# Example client code to query the deployed model
import requests

response = requests.post(
    "http://localhost:8000/generate",
    json={"prompt": "Artificial intelligence is", "max_length": 100}
)
print(response.json())

```

This deployment uses Ray Serve to create a scalable inference service. Ray Serve handles the complexity of scaling, load balancing, and resource management,

Real-World Applications and Case Studies of Ray

Ray has gained significant traction in various industries due to its ability to scale AI/ML workloads efficiently. Here are some notable real-world applications and case studies:

Large-Scale AI Model Training (OpenAI, Uber, and Meta)

- **OpenAI** used Ray to scale reinforcement learning for training AI agents like Dota 2 bots.
- **Uber's Michelangelo** leverages Ray for distributed hyperparameter tuning and model training at scale.
- **Meta (Facebook)** employs Ray to optimize large-scale deep learning workflows.

Financial Services and Fraud Detection (Ant Group, JP Morgan, and Goldman Sachs)

- **Ant Group** (Alibaba's fintech arm) integrates Ray for real-time fraud detection and risk assessment.
- **JP Morgan** and **Goldman Sachs** use Ray to accelerate financial modeling, risk analysis, and algorithmic trading strategies.

Autonomous Vehicles and Robotics (NVIDIA, Waymo, and Tesla)

- **NVIDIA** utilizes Ray for reinforcement learning-based autonomous driving simulations.

- **Waymo** and **Tesla** employ Ray to train self-driving car models with large-scale sensor data processing.

Healthcare and Drug Discovery (DeepMind, Genentech, and AstraZeneca)

- **DeepMind** leverages Ray for protein folding simulations and AI-driven medical research.
- **Genentech** and **AstraZeneca** use Ray in AI-driven drug discovery, accelerating computational biology and genomics research.

Large-Scale Recommendation Systems (Netflix, TikTok, and Amazon)

- **Netflix** employs Ray to power personalized content recommendations and A/B testing.
- **TikTok** scales recommendation models with Ray to improve video suggestions in real time.
- **Amazon** enhances its recommendation algorithms and e-commerce search using Ray's distributed computing capabilities.

Cloud & AI Infrastructure (Google Cloud, AWS, and Microsoft Azure)

- **Google Cloud Vertex AI** integrates Ray for scalable machine learning model training.
- **AWS SageMaker** supports Ray for distributed hyperparameter tuning.
- **Microsoft Azure** utilizes Ray for optimizing AI and machine learning services.

Ray at OpenAI: Powering Large Language Models

One of the most notable users of Ray is OpenAI, which has leveraged the framework for training its large language models, including ChatGPT. According to reports, Ray was key in enabling OpenAI to enhance its ability to train large models efficiently.

Before adopting Ray, OpenAI used a collection of custom tools to develop early models. However, as the limitations of this approach became apparent, the company switched to Ray. OpenAI's president, Greg Brockman, highlighted this transition at the Ray Summit.

The key advantage that Ray provides for LLM training is the ability to run the same code on both a developer's laptop and a massive distributed cluster. This capability becomes increasingly important as models grow in size and complexity.

Advanced Ray Features and Best Practices

Let us now explore advanced ray features and best practices:

Memory Management in Distributed Applications

Efficient memory management is crucial when working with large-scale ML workloads:

- **Object Spilling:** Ray automatically spills objects to disk when memory pressure is high. Configure spilling thresholds appropriately for your workload:

```
ray.init(  
    object_store_memory=10 * 10**9, # 10 GB  
    _memory_monitor_refresh_ms=100, # Check memory usage every 100ms  
)
```

[Copy Code](#)

- **Reference Management:** Explicitly delete references to large objects when no longer needed:

[Copy Code](#)

```
# Create a large object
data_ref = ray.put(large_dataset)

# Use the reference
result_ref = process_data.remote(data_ref)
result = ray.get(result_ref)

# Delete the reference when done
del data_ref
```

- **Streaming Data Processing:** For very large datasets, use Ray Data's streaming capabilities instead of loading everything into memory:

[Copy Code](#)

```
import ray
dataset = ray.data.read_csv("s3://bucket/large_dataset/*.csv")

# Process the dataset in batches without loading everything
for batch in dataset.iter_batches():
    # Process each batch
    process_batch(batch)
```

Debugging Distributed Applications

Debugging distributed applications can be challenging. Ray provides several tools to help:

- **Ray Dashboard:** Provides visibility into task execution, actor states, and resource usage:

[Copy Code](#)

```
# Start Ray with the dashboard enabled
ray.init(dashboard_host="0.0.0.0")
# Access the dashboard at http://<your-ip>:8265
```

- **Detailed Logging:** Use Ray's logging utilities to capture logs from all workers:

[Copy Code](#)

```
import ray
import logging

# Configure logging
ray.init(logging_level=logging.INFO)
```

```
@ray.remote
def task_with_logging():
    logger = logging.getLogger("ray")
    logger.info("This message will be captured in Ray's logs")
    return "Task completed"
```

- **Exception Handling:** Ray propagates exceptions from remote tasks back to the driver:

```
@ray.remote
def task_that_might_fail(x):
    if x < 0:
        raise ValueError("x must be non-negative")
    return x * x

# This will raise the ValueError in the driver
try:
    result = ray.get(task_that_might_fail.remote(-1))
except ValueError as e:
    print(f"Caught exception: {e}")
```

[Copy Code](#)

Ray vs. Other Distributed Computing Frameworks

We will now look in Ray vs. Other Distributed computing frameworks:

Ray vs. Dask

Both Ray and Dask are Python-native distributed computing frameworks, but they have different focuses:

- **Programming Model:** Ray's task and actor model provides more flexibility compared to Dask's task graph approach.
- **ML/AI Focus:** Ray has specialized libraries for ML (Train, Tune, Serve), while Dask focuses more on data processing.
- **Data Processing:** Dask has deeper integration with PyData ecosystem (NumPy, Pandas).

- **Performance:** Ray typically shows better performance for fine-grained tasks and dynamic workloads.

When to choose Ray over Dask:

- For ML-specific workloads (training, hyperparameter tuning, model serving)
- When you need the actor programming model for stateful computation
- For highly dynamic task graphs that change during execution

Ray vs. Apache Spark

Ray and Apache Spark serve different primary use cases:

- **Language Support:** Ray is Python-first, while Spark is JVM-based with Python bindings.
- **Use Cases:** Spark excels at batch data processing, while Ray is designed for ML/AI workloads.
- **Iteration Speed:** Ray offers faster iteration for ML experiments than Spark.
- **Programming Model:** Ray's model is more flexible than Spark's RDD/DataFrame abstractions.

When to choose Ray over Spark:

- For Python-native ML workflows
- When you need fine-grained task scheduling
- For interactive development and fast iteration cycles
- When building complex applications that mix batch and online processing

Ray vs. Kubernetes + Custom ML Code

While Kubernetes can be used to orchestrate ML workloads:

- **Abstraction Level:** Ray provides higher-level abstractions specific to ML/AI than Kubernetes.
- **Development Experience:** Ray offers a more seamless development experience without requiring knowledge of containers and YAML.
- **Integration:** Ray can run on Kubernetes, combining the strengths of both systems.

When to choose Ray over raw Kubernetes:

- To avoid the complexity of container orchestration
- For a more integrated ML development experience
- When you want to focus on algorithms rather than infrastructure

Reference: [Ray docs](#)

Conclusion

Ray has emerged as a critical tool for scaling AI and ML workloads, from research prototypes to production systems. Its intuitive programming model, combined with specialized libraries for training, tuning, and serving, makes it an attractive choice for organizations looking to scale their AI efforts efficiently. Ray provides a path to scale that doesn't require rewriting existing code or mastering complex distributed systems concepts.

By understanding Ray's core concepts, libraries, and best practices outlined in this guide, developers and data scientists can leverage distributed computing to tackle problems that would be infeasible on a single machine, opening up new possibilities in AI and ML development.

Whether you're training large language models, optimizing hyperparameters, serving models at scale, or processing massive datasets, Ray provides the tools and abstractions to make distributed computing accessible and productive. As the field continues to advance, Ray is positioned to play an increasingly important role in enabling the next generation of AI applications.

Key Takeaways

- Ray simplifies distributed computing for AI/ML by enabling seamless scaling from a single machine to a cluster with minimal code modifications.
- Ray's ecosystem (Train, Tune, Serve, Data) provides end-to-end solutions for distributed training, hyperparameter tuning, model serving, and data processing.
- Ray's task and actor-based programming model makes parallelization intuitive, transforming Python applications into scalable distributed workloads.
- It optimizes resource management through efficient scheduling, memory management, and automatic scaling across CPU/GPU clusters.
- Real-world AI applications at scale, including LLM fine-tuning, reinforcement learning, and large-scale data processing.

The media shown in this article is not owned by Analytics Vidhya and is used at the Author's discretion.



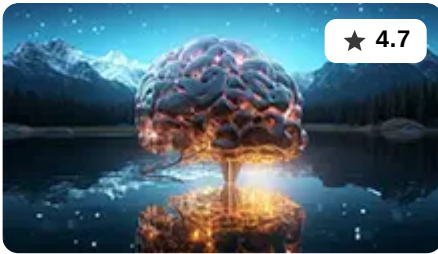
Krishnaveni Ponna

Hello! I'm a passionate AI and Machine Learning enthusiast currently exploring the exciting realms of Deep Learning, MLOps, and Generative AI. I enjoy diving into new projects and uncovering innovative techniques that push the boundaries of

technology. I'll be sharing guides, tutorials, and project insights based on my own experiences, so we can learn and grow together. Join me on this journey as we explore, experiment, and build amazing solutions in the world of AI and beyond!

[Advanced](#)[Artificial Intelligence](#)[Guide](#)[Machine Learning](#)

Free Courses



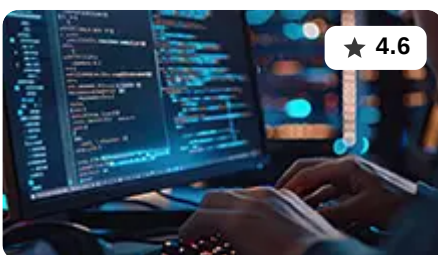
Generative AI - A Way of Life

Explore Generative AI for beginners: create text and images, use top AI tools, learn practical skills, and ethics.



Getting Started with Large Language Models

Master Large Language Models (LLMs) with this course, offering clear guidance in NLP and model training made simple.



Building LLM Applications using Prompt Engineering

This free course guides you on building LLM apps, mastering prompt engineering, and developing chatbots with enterprise data.



Improving Real World RAG Systems: Key Challenges & Practical Solutions

Explore practical solutions, advanced retrieval strategies, and agentic RAG systems to improve context, relevance, and accuracy in AI-driven applications.



Microsoft Excel: Formulas & Functions

Master MS Excel for data analysis with key formulas, functions, and LookUp tools in this comprehensive course.

RECOMMENDED ARTICLES

[Top 7 Frameworks for Building AI Agents in 2025](#)

[Top 10 Must Use AI Tools for Data Analysis \[202...](#)

[Is Python Ray the Fast Lane to Distributed Comp...](#)

[Ultimate guide to handle Big Datasets for Machi...](#)

[Understanding Dask in Depth](#)

[Optimizing AI Performance: A Guide to Efficient...](#)

[An End-to-End Starter Guide on Apache Spark and...](#)

[Getting Started with Apache Arrow](#)

[Futurist Ray Kurzweil Claims Humans Will Achiev...](#)

[A Comprehensive Guide to Smallpond](#)

Responses From Readers

What are your thoughts?...

Submit reply

Write for us →

Write, captivate, and earn accolades and rewards for your work

- Reach a Global Audience
- Get Expert Feedback
- Build Your Brand & Audience
- Cash In on Your Knowledge
- Join a Thriving Community
- Level Up Your Data Science Game



Company

- About Us
- Contact Us
- Careers

Learn

- Free Courses
- AI&ML Program
- Pinnacle Plus Program
- Agentic AI Program

Contribute

- Become an Author
- Become a Speaker
- Become a Mentor

Discover

- Blogs
- Expert Sessions
- Learning Paths
- Comprehensive Guides

Engage

- Community
- Hackathons
- Events
- Podcasts

Enterprise

- Our Offerings
- Trainings
- Data Culture

[Terms & conditions](#) • [Refund Policy](#) • [Privacy Policy](#) • [Cookies Policy](#) © Analytics
Vidhya 2025.All rights reserved.